

Questão 1 [2 pontos]:

Mostre como remover todos os elementos duplicados de um vetor em $O(n \log n)$, adaptando o algoritmo Mergesort.

Questão 2 [2 pontos]:

Dado um vetor A com n entradas, com cada entrada um número distinto. Sabe-se que a sequência de valores $A[1], A[2], \dots, A[n]$ é unimodal: ou seja, existe índice p entre 1 e n , os valores nas entradas do vetor aumentam até a posição p e, em seguida, reduzem, o resto do caminho, até que a posição n . Deseja-se encontrar o "ponto de máximo" p . Mostre como encontrar p através de um algoritmo $O(\log n)$. (Verso)

Questão 3 [2 pontos]:

Você tem uma sequência de números inteiros, e deseja encontrar o valor máximo e mínimo presentes nessa sequência. Projete um algoritmo eficiente de Divisão e Conquista para encontrar esses dois valores, explicando como sua abordagem funciona e analisando sua complexidade.

Questão 4 [2 pontos]:

Um aluno de PAA está planejando fazer uma viagem de carro e tem um tanque de gasolina com capacidade para L litros. Ele possui também tem uma lista de postos de gasolina ao longo do caminho, cada um com uma distância da sua posição atual e um preço por litro. Projete um algoritmo guloso para determinar a rota que minimize o custo total da viagem.

Questão 5 [2 ponto]:

Considere o seguinte algoritmo guloso para o CAIXEIRO VIAJANTE. Mostre que ele não é exato, isto é, não garante retornar a solução ótima. Para isso use um contra-exemplo.

Algoritmo 1 (Algoritmo do vizinho mais próximo)

1. Inicialize todos os vértices como não visitados.
2. Selecione um vértice arbitrário, defina-o como o vértice atual u . Marque u como visitado.
3. Descubra a aresta mais curta conectando o vértice atual u e um vértice não visitado v .
4. Defina v como o vértice atual u . Marque v como visitado.
5. Se todos os vértices do domínio forem visitados, então termine. Caso contrário, vá para o passo 3.

Questão 6 [2 pontos]:

Um ladrão entra em uma loja e vê um conjunto I de n itens, $I = \{a_1, a_2, \dots, a_n\}$. Cada item tem um peso associado w_i e um valor v_i . Idealmente, o ladrão gostaria de roubar tudo para obter o máximo benefício. No entanto, há muito que ele pode carregar. O ladrão tem uma mochila com capacidade K . O ladrão agora tem que determinar quais itens roubar para que seu peso total não exceda K e seu valor total seja máximo.

1. Suponha que os itens sejam tais que uma fração de um item possa ser retirada, ou seja, o ladrão pode pegar alguma parte de um item (por exemplo, $0,4 w_i$ de a_i) e deixar a parte restante. Isso é chamado de Problema da Mochila Fracionada. Seja $A \subseteq I$ o subconjunto de itens que o ladrão rouba. Elabore um algoritmo guloso $O(n \log n)$ para encontrar o subconjunto A tal que $\sum_{a_i \in A} w_i < K$ e $\sum_{a_i \in A} v_i$ é máximo.
2. Suponha que os itens só possam ser retirados como um todo, ou seja, o ladrão só pode pegar ou deixar um item; ele não pode pegar uma fração de um item. Isso é chamado de Problema da Mochila Binária (ou Mochila 0-1). Seu algoritmo guloso para a versão fracionária do problema (pergunta anterior) ainda encontra uma solução ótima? Justifique sua resposta.

Questão 01. (Desconsiderar) ~~Questão esta na última folha.~~

~~Dado o que foi pedido na questão, podemos demonstrar a partir do seguinte:~~

~~def removerDuplicados (Vetor):~~

~~VetorOrdenado = Ordenação (Vetor) # Ordenação do Vetor em $O(n \cdot \log n)$ (MergeSort)~~

~~resultado = []~~

~~aux = VetorOrdenado[0]~~

~~resultado.append(aux)~~

~~for v in VetorOrdenado[1:]: # Eliminação em $O(n)$.~~

~~if v != aux:~~

~~resultado.append(v)~~

~~aux = v~~

~~return resultado~~

~~Logo, o algoritmo acima é $O(n \log n)$ e elimina todos os elementos duplicados de um~~

~~vetor.~~

Questão 02.

Dado o que foi pedido, vamos ter o seguinte algoritmo:

def maximo (Vetor, i, n):

if i == n:

return i

if i == n-1:

if Vetor[i] >= Vetor[n]:

return i

else

return n

m = (i+n)/2

if Vetor[m] > Vetor[m-1] and Vetor[m] > Vetor[m+1]:

return m

elif Vetor[m] > Vetor[m+1] and Vetor[m] < Vetor[m-1]:

return maximo (Vetor, i, m-1)

else

return maximo (Vetor, m+1, n)

2.0

03.

partir do que foi pedido, Vamos obter o seguinte algoritmo:

def encontrarMaxMin(Vetor):

if len(Vetor) == 0:

return

#①

elif len(Vetor) == 1:

return [Vetor[0], Vetor[0]]

elif len(Vetor) == 2:

if Vetor[0] > Vetor[1]:

return [Vetor[1], Vetor[0]]

else

return [Vetor[0], Vetor[1]]

else:

m = len(Vetor) / 2

#②

Va = encontrarMaxMin(Vetor[:m])

Vb = encontrarMaxMin(Vetor[m+1:])

Vresultado = []

if Va[0] < Vb[0]:

#③

Vresultado[0] = Va[0]

else

Vresultado[0] = Vb[0]

if Va[1] > Vb[1]:

Vresultado[1] = Va[1]

else

Vresultado[1] = Vb[1]

A abordagem foi:

① Definição das cores bases.

② Divisão do vetor no meio para realizar a procura da divisão

③ Mesclagem da divisão para realizar conquista dos resultados.

Seu assim, o alg. retorna uma lista de por ordem com valor mínimo e máximo

em ordem. Por fim, temos que esse algoritmo encontrarMaxMin é $O(\log n)$.

Questão 04.

Dado o que foi pedido, temos o seguinte algoritmo:

1. Ordene a lista com a divisão da distância por valor e ordene a mais perto os destino.

2. Selecione a posição atual e siga até a menor relação da lista

3. A chegada, retire o elemento da relação da lista e atualize sua posição atual.

4. Chegou ao destino? Finalize, Ainda não? Retorne ao passo 2.

Pseudo-código

0,5

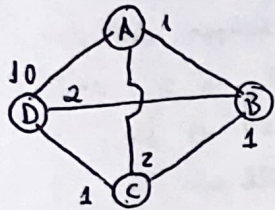
Não verifica

se o carro consegue chegar ao destino.

Questão 05

Dado o problema do Caminho Viciante, os avaliadores o seguinte código (Vizinhos mais próximos), apresentamos o contraexemplo abaixo:

Dado o grafo



A partir do grafo descrito. A partir do algoritmo 1, supondo que iniciemos com nó A. Ele irá ter a seguinte execução:

$A \rightarrow B \rightarrow C \rightarrow D \rightarrow A$. Tal caminho,

tem um custo de 13.

Seja a solução ótima, a seguinte execução:

$A \rightarrow B \rightarrow D \rightarrow C \rightarrow A$. Tal caminho,

tem um custo de 6.

Logo, a partir do contraexemplo temos que Alg. 1 não é exato.

Questão 06

~~def removerDuplados (Vetor, i, f)~~

~~if len(Vetor) == 1:~~

~~return~~

Questão 06

1. Temos o seguinte alg. guloso:

1. Dado o conjunto de I itens, ordene de forma decrescente sob o critério $\frac{\text{valor}}{\text{peso}} = \frac{V_i}{w_i}$.
2. Em seguida, coloque o primeiro item do conjunto ordenado.
3. Continue até que a mochila atinja a capacidade máxima, após isso finalize.

Dessa forma, teremos a resolução do problema da mochila fracionada.

05

Não coloca frações dos itens